

Show HN — Mesedi launch draft

Status: Draft. Not yet submitted.

Gated on: #120 (Stripe self-serve), #133 (design-partner feedback).

Submission window target: Tuesday or Wednesday, 8:30–10:00 AM Eastern. Avoid Mondays (Show HN backlog from weekend) and Fridays (lower traffic).

Title

Show HN: Mesedi – Alert-first observability for AI agents (8 detectors, MIT)

77 characters including "Show HN:". HN truncates at ~80 on the front page. Alternates considered and rejected:

- *Show HN: Mesedi – PagerDuty for AI agents* — too marketing-flavored, gets downvoted as buzzwordy.
- *Show HN: Mesedi – Detect 8 AI-agent failure classes* — true but doesn't convey the architectural wedge.
- *Show HN: Mesedi – Open-source agent observability with playbooks* — buries the lede; "alert-first" is the contested word that draws engagement.

Body

Plain text, no markdown. HN strips most formatting. Paragraph breaks via blank lines only. Word count $\approx$ 640, reading time $\approx$ 2:30 — under the 800-word soft ceiling where HN starts skimming.

Hi HN. I've been building Mesedi for about six months. It's an observability tool specifically for AI agents. The wedge versus existing tools: most AI observability today is trace-first. LangSmith, Langfuse, Axiom. The eight classes are: crashes, identical-call loops, similar-call loops (the agent keeps rewording the same question), etc. A few things this would have caught that made the news: Air Canada's chatbot inventing a bereavement discount. What's working: Python and TypeScript SDKs are on PyPI and npm, one-line wrap around an existing agent. What's not working yet, to be honest: Stripe self-serve checkout just landed and I'm still finding bugs. Pricing: self-host is free, MIT, no rate limits. Cloud Hobby is \$0 with 5K execs/month, Cloud Pro is \$100. I'd love feedback on three things specifically: (1) is the detector set the right eight, or am I missing something? (2) is the alerting good? (3) is the demo good? Demo: <https://mesedi.vercel.app> (signup is real and ~30 seconds, no card for Hobby). Repo: <https://github.com/mesedi/mesedi>

Pre-drafted Q&A; replies

Top-of-thread questions that always come up on Show HN posts for dev tools. Pre-drafted so you can paste-and-tweak in the first 4–6 hours when engagement matters most. Tone: honest, brief, technical, no superlatives.

Q: How is this different from Langfuse?

Langfuse is a great tracing platform — universal LLM call capture, prompt management, evals, custom dashboards on top of your spans. If you want to instrument your LLM calls and decide later what to alert on, Langfuse is the right tool.

Mesedi is doing something narrower. It runs eight specific failure-class detectors against the event stream (loops, cost spikes, prompt-injection patterns, validator rejections, output drift, etc.) and fires once-per-pattern alerts with a canonical fix description attached. There is no general-purpose dashboard-builder. If your need is "tell me when the agent is broken in a way I care about, with a recommended fix," Mesedi is closer to the shape. Both are MIT — there's no reason you couldn't run both. Langfuse for forensics, Mesedi for paging.

Q: How does "clustering" actually work — is it ML?

No, it's deliberately not ML for the v1 detectors. Each detector emits a (failure_class, signature) pair where the signature is either an exact string (e.g., the prompt-injection pattern name, or the failed validator name) or a deterministic hash of the relevant features (e.g., `identical_call_<hash(model + user_message)>` for the loop detector). Two failures with the same (class, signature) get grouped into the same failure_group row, which is upserted on each occurrence with last_seen updated.

The reason it's not ML is calibration: a customer who has never used Mesedi shouldn't need a baseline period before the first alert lands. Hash-based clustering means the first occurrence of any pattern fires immediately, the second occurrence increments a counter, and you get paged exactly once. Drift detection is the one detector with a learned baseline (character 3-gram cosine against the project's first ~1000 outputs), which is fine because drift inherently requires "drift from what."

Q: What does the SDK actually do? Does it add latency?

The SDK is a one-line wrapper. In Python: `@mesedi.wrap` on your agent function. In TypeScript: `wrap(async (input) => ...)`. The wrapper records start/end timestamps, captures any LLM-call and tool-call events that happen inside (we patch the popular OpenAI/Anthropic/etc. clients), and posts the resulting event batch to the backend asynchronously after the agent returns. Hot-path overhead is the time to push events into an in-process queue — sub-millisecond. Network I/O happens on a background thread.

If the backend is unreachable, the SDK degrades to logging warnings and dropping events; the agent itself never errors because of a Mesedi outage. That's a deliberate design choice — observability tools that take down production traffic are a worse failure mode than missing telemetry.

Q: Why Go for the backend? Why SQLite?

Go because the binary is single-file, no runtime dependency, and cross-compiles to any Linux box. The "self-host on a Hetzner VM" path is: `scp ./mesedi-backend; ./mesedi-backend`; done. I wanted that to be true.

SQLite because v1 telemetry volume fits comfortably on one writer, the WAL journal mode handles concurrent reads fine, and migrations are embedded in the binary via go:embed. There's no Postgres setup, no managed-database bill, no migration tooling to ship. The store interface is abstracted, so the Postgres migration (queued for Phase 4) is a one-day swap — but until customer load actually justifies it, SQLite is the right answer.

Q: How does drift detection work?

Drift detection runs character 3-gram cosine similarity between each agent output and a baseline computed from the project's first ~1000 outputs. When cosine drops below a configured threshold (default 0.65) on more than a configured fraction of recent outputs, a drift/lexical_drift_<hash> failure_group fires. Character 3-grams (not word tokens, not embeddings) because they're language-agnostic, robust to small phrasing changes, and don't require running a model.

There's also a drift/new_model:<name> detector that fires the first time an execution in your project uses a model name the project has never seen before — catches accidental model swaps (the "we upgraded to gpt-4-turbo and now nothing works" case). Both are simple by design.

Q: Privacy — do you see our prompts?

If you self-host (MIT, single-binary Go), no part of your event stream leaves your network. The backend embeds the dashboard, the migrations, and the playbook content. You run it on your hardware, talk to it from your SDK, done.

If you use the hosted Cloud service, event payloads are stored in encrypted SQLite on Fly.io (US region) with 7-day retention on Hobby, 30-day on Pro. The SDKs include a redact= argument that lets you specify which event fields to strip client-side before they leave your network (typical use: redact the user_message payload but keep the model name, token counts, and timing). Validator and tool-failure events capture event metadata, not the full prompt body, by default.

If you need stricter guarantees than "trust the operator," self-hosting is the answer. The source is right there.

Q: How is this not just regex on stderr?

A few of the detectors are pattern-based — the prompt-injection detector matches on a set of curated regexes (instruction-tag sentinels, named jailbreak phrases, role-override syntactic shapes) maintained in detectors/injection.go. That's intentional: prompt injection is a class where the literal token pattern is the signal. Adding new patterns is a one-line PR.

The other seven detectors aren't regex. Identical-call clusters on a hash of (model, normalized_user_message) across consecutive llm_call events in one execution. Time-budget and step-count are deterministic threshold checks. Cost-velocity is a rolling-window rate calculation. Validator failures observe payload.passed=false on validator events the SDK forwards. Drift is character 3-gram cosine vs baseline. The detector code lives in backend/internal/detectors/ if you want to read it before deciding.

Q: Self-hosting — how hard?

Three commands. git clone github.com/mesedi-ai/mesedi, cd mesedi/backend && go build -o mesedi-backend ./cmd/api, ./mesedi-backend --port 8080 --db-url file:./data.db. The binary embeds the dashboard, the migrations, and the playbook content. Point your SDK at http://your-host:8080 via the MESEDI_API_URL env var. There's

no separate frontend deploy, no Postgres, no Redis, no message queue. SQLite handles the data, embedded HTTP serves the dashboard, the same binary runs the webhook dispatcher and the detector pipeline.

If you want it behind TLS, put Caddy or nginx in front. If you want HA, run two instances behind a load balancer once Postgres lands in Phase 4 — for v1 it's single-writer SQLite, so one instance is the supported topology.

Q: Does it work with LangChain / CrewAI / Vercel AI SDK?

Yes. There are first-class adapters for all three: `mesedi.frameworks.langchain.MesediCallbackHandler` (LangChain Python), `mesedi.frameworks.crewai.attach()` (CrewAI Python), and `import { mesediMiddleware } from 'mesedi/vercel'` (Vercel AI SDK TypeScript). The adapters subscribe to the framework's native callback/middleware hooks and emit Mesedi events, so you don't have to manually wrap each step.

For anything else, the `wrap()` primitive plus `events.emit()` covers it. The Python and TS SDKs both accept a custom event emitter you can hook into your own agent framework.

Q: "Tier 1 Playbooks" — implies tiers 2 and 3. What are they?

Tier 1 is what ships today: per-signature canonical markdown describing what the pattern usually means and the standard remediation, written by the maintainer (me) and shipped in the repo. Zero liability — Mesedi doesn't take any action; the engineer reading the alert decides what to do.

Tier 2 (planned, no committed date): customer-curated playbooks. Same markdown shape, but written by your team for your project. Survives across deploys via the storage layer.

Tier 3 (research, no committed date): suggested remediations grounded in your codebase. The hard problem: doing this responsibly without auto-applying fixes that turn a \$40 incident into a \$40,000 incident. Not on the v1 critical path.

Q: Why \$29 when Langfuse is also \$29?

Because I'm not trying to undercut on price. Mesedi is doing a different job (alert-first detection + playbooks, not trace-first capture + dashboards), so the comparison isn't apples-to-apples on features. If you compare on price, the value comes from the eight detectors and the canonical playbooks shipping out of the box — those don't exist in Langfuse, and recreating them would mean writing your own alert logic against the Langfuse trace store.

If you decide \$29 isn't worth it because trace-first fits your workflow better, that's a fair call — Langfuse is excellent at the job it does. The honest answer is they're not direct competitors even though they overlap on surface.

Q: How do you handle multi-agent / hierarchical systems?

Each `wrap()`-ed function call is one execution. Nested calls (parent agent calls child agent calls tool) record `parent_execution_id` on the child, so the dashboard shows the hierarchy. Failure-class detection runs per-execution but the grouping rolls up — if an identical-call loop fires in the child but not the parent, you see it in the child's failure-group with the parent linked.

No special "multi-agent framework" support beyond that; CrewAI's role-based agents and LangGraph's state machines both work, you just get one execution per agent step and the hierarchy in the dashboard.

Submission-day checklist

1. Pick a Tuesday or Wednesday, 8:30–10:00 AM Eastern.
2. Verify mesedi.vercel.app is up and signup works end-to-end. Do a fresh signup from a clean browser before posting.
3. Verify github.com/mesedi-ai/mesedi is public and the README is current. Lock the repo from force-pushes for the day.
4. Pre-warm: one tweet / Mastodon / LinkedIn post 30 minutes before HN submission, link to mesedi.vercel.app, no mention of HN itself (HN downvotes anything that smells like brigading).
5. Submit to news.ycombinator.com. Title goes in the title field, body in the text field. Leave the URL field BLANK — text posts work better for Show HN than URL submissions.
6. Refresh comments every 5–10 minutes for the first 6 hours. Reply within 15 minutes per question. The pre-drafted answers above are starting points, not scripts — read each question and adapt.
7. If a question has been asked 3+ times, edit the post body to add a short FAQ at the bottom. HN appreciates this; it's seen as good citizenship.
8. Don't argue with downvoters. Don't argue with anyone, actually — answer the question, link to evidence, move on.
9. Take notes on the questions you didn't have a good answer for. Those go into the product backlog and the next iteration of the marketing copy.

Failure mode planning

If the post hits the front page:

- Vercel and Fly handle the traffic fine; both are autoscaling. The backend is `min_machines_running = 1` with `auto_start_machines = true`, can handle ~200 concurrent requests per instance per the `fly.toml` soft limit.
- The signup endpoint has in-process IP rate limiting; HN traffic from one ASN shouldn't bypass it.
- The synthetic-org launchd job will keep producing dogfood traffic during the spike; might want to pause it for the day to keep the deliveries page clean for any HN visitor who lands on it.

If the post sinks (most do):

- Don't repost. HN auto-detects reposts and shadow-deletes them.
- Note what didn't land. The next launch vehicle is Product Hunt (#132) and the long-tail one is design-partner outreach (#133).